

Automatización Inteligente BPO: Diseño y Arquitectura de Microservicio con IA

Juan Sebastian Rodriguez Salazar
Seguros Bolívar
Caso de Estudio IA - Profesional Senior
Arquitectura Cloud y Modelos de Lenguaje

Abstract

Este documento técnico detalla el diseño, arquitectura y estrategias de escalabilidad para el microservicio backend automatizado desarrollado para una empresa BPO. La solución centraliza el procesamiento de tickets (Petición, Quejas, Reclamos) para múltiples clientes utilizando Inteligencia Artificial, sin necesidad de duplicar código.

Basado en los principios de la Arquitectura Hexagonal y garantizando el principio Open/Closed, la regla de negocio es extraída a una capa de metadatos (YAML). Se implementaron patrones como *Chain of Responsibility* para el ciclo de vida de 6 etapas de la PQR y *Strategy* para el enrutamiento multi-cliente. Para garantizar un SLA crítico, se diseñó un *Proxy de Fallback* asíncrono que alterna entre el modelo primario (Groq LPU) y el secundario (Google Gemini) ante cualquier anomalía de red o cuota.

Aquí se expone la metodología empleada, la superación de retos de seguridad, la recolección de métricas de resiliencia y el camino hacia un despliegue nativo en la nube utilizando contenedores inmutables en Google Cloud Run y gestión de secretos criptográficos.

1 Introducción

La gestión manual de solicitudes PQR de texto libre (omnichannel) es uno de los mayores cuellos de botella operativos en una empresa BPO (Business Process Outsourcing). El reto radica en interpretar el lenguaje natural, abstraer entidades clave e inyectar el resultado estructurado en el CRM de la compañía cliente.

El microservicio aquí presentado automatiza este flujo mediante un pipeline de seis pasos lógicos: *Validar, Clasificar, Priorizar, Justificar, Enrutar y Gestionar la plataforma externa*. Para lograrlo, se emplea un clúster de Inteligencia Artificial (LLM) que procesa la semántica del usuario apoyado por directrices estructuradas a nivel de prompts deterministas.

A continuación, se describen las decisiones ingenieriles, los patrones matemáticos de código adoptados (Arquitectura Hexagonal), la estrategia de Cloud Computing seleccionada (GCP) para el despliegue del contenedor actual y cómo se resolvieron problemas de consistencia semántica en los requerimientos del negocio.

2 Ingeniería de Contexto y Agentic Workflows

Acorde a las vanguardias de la ingeniería de software, el ciclo de vida de este desarrollo implementó metodologías de desarrollo asistido por IA (*Agentic Workflows*).

Se construyó un marco de contexto estricto (*Context Engineering*) definiendo contratos *Pydantic*, interfaces abstractas y diagramas de estado. Esta base documental rigurosa fue utilizada para orquestar agentes generativos (mediante el entorno *Anti-gravity*), logrando materializar el código de manera predecible y determinista.

Este enfoque multiplicó la velocidad de entrega, demostrando que el manejo de las herramientas de AI para el desarrollo y producir código que respete los principios SOLID y la Arquitectura Hexagonal.

3 Diseño Arquitectónico y Metodología

La solución fue concebida iterativamente con el objetivo principal de aislar las reglas de negocio del framework web (FastAPI) y de los proveedores del LLM, habilitando expansiones futuras de manera inocua.

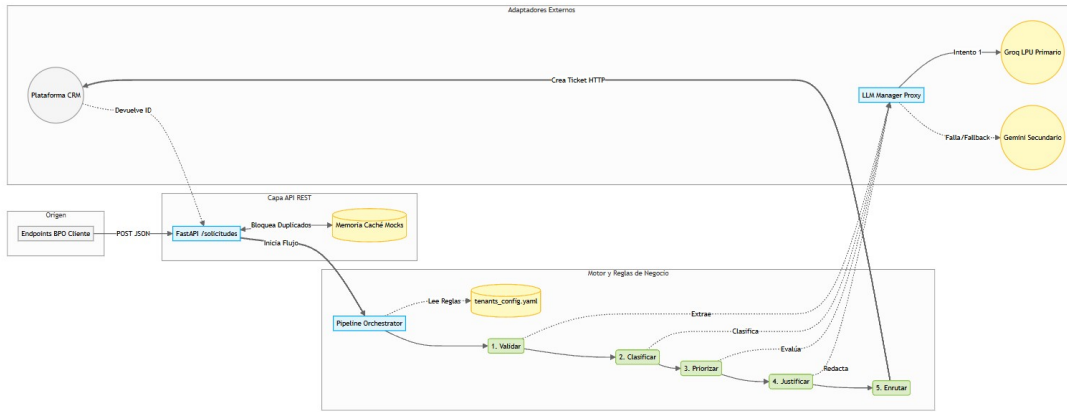


Figure 1: Diagrama de Arquitectura Hexagonal, Inversión de Dependencias y orquestación BPO.

3.1 Abstracción Multi-Tenant y Patrones

El principal reto era cumplir el principio *Open/Closed (SOLID)*. Para evitar modificar código fuente ante el ingreso de una nueva compañía al BPO, se centralizaron las reglas de negocio en el archivo `data/tenants_config.yaml`. Este archivo contiene los campos requeridos, las categorías y los enrutamiento por cruce de las prioridades.

La lógica arquitectónica reposa sobre dos pilares:

- **Chain of Responsibility:** El `PipelineOrchestrator` orquesta el ciclo de vida. Incluye lógica de *Early Exit*: si en el primer paso el LLM advierte que faltan datos (p. ej., el cliente no da su cédula), el pipeline corta el flujo protegiendo la plataforma y el gasto de IA.
- **Strategy Pattern:** La `StrategyFactory` asigna un comportamiento de negocio según el inquilino. Mientras la mayoría de clientes delegan el cálculo de prioridad en el LLM (`DefaultStrategy`), clientes especiales (como "Mensajería del Valle") utilizan un puente REST mockeado inyectando su lógica propia de negocio a través de `MensajeriaValleStrategy`.

3.2 Tolerancia a Fallos Multimodelo (Fallback)

Previendo la inestabilidad inherente de las integraciones de terceros y las penalizaciones por *Rate Limits* (HTTP 429), se introdujo una clase `LLMManager`. Este *Proxy Adapter* enmascara los

errores del modelo rápido primario (Groq) reintentando de forma autónoma y silenciosa con un motor *Fallback* de contingencia (Google Gemini 2.5). Las pruebas demostraron viabilidad absoluta: ante la caída total del modelo primario, las subrutinas de la PQR procedieron hasta el final arrojando respuestas de éxito (HTTP 200).

4 Estrategia Cloud, Seguridad y Docker

Trascendiendo el funcionamiento lógico, un microservicio corporativo requiere robustez y protección contra vulnerabilidades cibernéticas, empaquetamiento confiable y diseño de nube de alta disponibilidad.

4.1 Contenerización Serverless (GCP Cloud Run)

El código entregado ya cuenta con garantía de inmutabilidad al incluir su `Dockerfile` y `docker-compose.yml`. Esto aísla la ejecución independientemente de la máquina del desarrollador.

En lugar de aprovisionar clusters de alta complejidad (ej. Kubernetes), el contenedor actual es el candidato óptimo para su despliegue en **Google Cloud Run**. Esta plataforma serverless facilita el escalamiento a cero (*scale-to-zero*) minimizando facturación en tiempos de inactividad, con capacidad de disparar paralelamente centenares de instancias cuando la carga transaccional del BPO así lo exija.

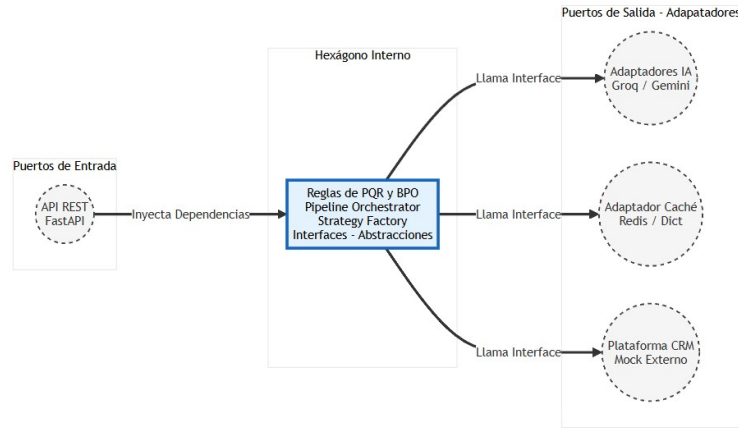


Figure 2: Diagrama de Arquitectura Hexagonal y Pipeline Asíncrono.

4.2 Seguridad, Auth y Secrets Manager

El cuidado estricto sobre las credenciales externas dictamina que parámetros confidenciales como `GROQ_API_KEY` o contraseñas de bases de datos no residen jamás en texto plano. En entorno local interactúan inyectándolas mediante variables `.env`; y mediante **Google Secret Manager** o AWS Secrets Manager en la nube productiva.

Asimismo, dado que la capa externa REST será consumida constantemente, se requiere prevenir escenarios de abuso (DDoS o robo de tokens AI). El pipeline proyecta blindarse mediante un middleware global con exigencia de JWT por el marco de **OAuth 2.0** restringido por inquilino.

5 Optimización y Bug-Fixing del Diseño

Una fortaleza cardinal en la ingeniería de software es refutar constructivamente la ambigüedad en los requerimientos pautados antes de que maduren en inconsistencias de bases de datos integradas.

5.1 Corrección Semántica de Esquemas

Durante la evaluación de los Mockups entregados en las directrices iniciales del Pdf, se notó una disonancia conceptual: el campo *numérico* del cliente emergía dentro del "tipo de id", como `solicitud.tipo_id_cliente: "102045678"`, y su género en el identificador `solicitud.id_cliente: "CC"`.

Para prevenir catástrofes de colisión y modelado

relacional, las clases en Pydantic V2 fueron corregidas. Se inyectaron *Reglas Estrictas de Extracción* deterministas en los prompts subyacentes del LLM. Ahora el LLM garantiza obligatoriamente colocar los valores numéricos bajo la llave de ID, salvaguardando la pureza de la ingesta de Data Engineering.

5.2 Idempotencia del Servicio Múltiple

La infraestructura incluye un `ICacheProvider` (`InMemoryCache`). En lugar de consumir tokens de IA en solicitudes que un usuario impaciente envía dos veces por segundo, se valida el Hash de petición (*Bono Idempotencia*). De migrarse a GCP, el caché volátil actual se vería reemplazado por **GCP Memorystore (Redis)** para cruzar validaciones de colisión idéntica a través de concurrencia multihilo de Cloud Run.

6 Conclusiones

Este proyecto abordó holísticamente la integración de automatismos LLM dentro de arquitecturas transaccionales resilientes. El acoplamiento estricto del patrón *Chain of Responsibility* dotado de la matriz multi-tenant (YAML) posibilitó resolver las 6 directivas de manera desacoplada.

La integración robusta del motor asíncrono secundario y falsificación de APIS (mocks transaccionales en `BPOExternalAdapter`) no solo brindaron tolerancia a caídas del BPO en la retención del log (*Pendiente por sincronización*) sino que aseguraron con suficiencia el sistema ante anomalías cloud imprevistas.

El microservicio PQR cumple así rigor empresarial, se halla empaquetado y perfilado de forma madura para escalamientos orgánicos *Serverless* en la ecosfera de Google Cloud.